

Guacamole on Amazon EKS

Infrastructure Architecture & IAM Permissions Reference

Region: us-east-1

EKS Cluster: guacamole (Kubernetes v1.35)

Source: playsoft-infra/aws_eks (Terraform + Kubernetes manifests)

Document date: 2026-07-24

This document describes the intended architecture of the Guacamole-on-EKS stack as defined in Terraform and the Kubernetes manifests, the purpose of each component, and the AWS IAM / Kubernetes permissions each part requires to operate. It is a design and permissions reference, independent of whether the stack is currently deployed.

1. Architecture Overview

The stack is organised in four layers: (1) an AWS network layer (VPC, subnets, gateways) that the cluster and its nodes run inside; (2) the Amazon EKS control plane and a managed EC2 node group that together form the Kubernetes cluster; (3) two IAM roles that grant the cluster's own components (kubelet, the EBS CSI driver, the AWS Load Balancer Controller) permission to call AWS APIs on their own behalf; and (4) the Kubernetes workloads in the **guacamole** namespace that make up the application itself — the Guacamole web UI, the guacd proxy, PostgreSQL, Redis, and a nightly backup job.

A useful way to read this document: every AWS-level component in sections 2–7 needs someone (a human IAM user, or an IAM role assumed by AWS) to have specific permissions before it can be created or can operate. Section 9 consolidates all of those permission requirements into one table.

2. Network Layer (VPC)

A dedicated VPC spans two Availability Zones, each with one public and one private subnet. Public subnets host the internet-facing Application Load Balancer and the single NAT Gateway; private subnets host the EKS worker nodes, which have no direct route to the internet.

Component	CIDR / Detail	Purpose
VPC	10.30.0.0/16	Isolated network for the entire cluster and its load balancer.
Public subnet (1a)	10.30.0.0/24	Hosts the NAT Gateway and one ALB elastic network interface.
Public subnet (1b)	10.30.1.0/24	Hosts a second ALB elastic network interface (multi-AZ ALB).
Private subnet (1a)	10.30.10.0/24	Hosts EKS worker node 1; routes outbound traffic via the NAT Gateway.
Private subnet (1b)	10.30.11.0/24	Hosts EKS worker node 2; routes outbound traffic via the NAT Gateway (cross-AZ).
Internet Gateway	—	Gives public subnets a route to/from the internet.
NAT Gateway	1, in subnet 1a	Lets private-subnet nodes reach the internet (image pulls, AWS APIs) without being reachable from it.
Route tables	public → IGW, private → NAT	Direct traffic per subnet type.

Permissions required to provision this layer

Standard EC2 networking permissions (create/describe/delete/tag on VPCs, subnets, internet gateways, NAT gateways, elastic IPs, and route tables). These are broad, commonly-granted EC2 permissions distinct from the narrower EKS- and IAM-specific permissions described later in this document.

3. EKS Control Plane

The control plane (API server, etcd, scheduler, controller-manager) is fully managed and operated by AWS — it does not run on any EC2 instance you can access, and there is no SSH or shell access to it under any circumstance.

Setting	Value
Cluster name	guacamole
Kubernetes version	1.35
Access mode	API (bootstrap-cluster-creator-admin-permissions)
Endpoint access	Public + private, allowed CIDR 0.0.0.0/0 (see Section 10)
IAM role used	eks-cluster-role (Section 5)

Permissions required

Permission	Why it's needed
eks:CreateCluster, DescribeCluster, UpdateClusterConfig, UpdateClusterVersion, DeleteCluster, TagResource / UntagResource	Scoped to the cluster's ARN. Lets the deploying user create and manage the control plane.
iam:PassRole	On eks-cluster-role, conditioned on iam:PassedToService = eks.amazonaws.com. Lets EKS assume that role for the control plane.
iam:CreateServiceLinkedRole	One-time, account-wide: AWS auto-creates AWSServiceRoleForAmazonEKS the first time any EKS cluster is created in the account.

4. Managed Node Group

A managed node group provides the EC2 instances that actually run pods. EKS handles provisioning, joining nodes to the cluster, and (within the configured range) replacing unhealthy instances.

Setting	Value
Name	jilani-k8s-eks-default
Instance type	t3.large
AMI	AL2023_x86_64_STANDARD
Capacity type	ON_DEMAND
Scaling	desired 2 / min 2 / max 4
Disk	40 GiB gp3 (root)
Placement	Both private subnets (one node per AZ)
SSH access	Key pair configured, security group allows :22 from admin_cidr — but nodes sit in private subnets with no bastion or SSM path, so this is currently unreachable in practice.

Permissions required

Permission	Why it's needed
eks:CreateNodegroup, DescribeNodegroup, UpdateNodegroupConfig, UpdateNodegroupVersion, DeleteNodegroup, ListNodegroups, TagResource / UntagResource	Scoped to the node group's ARN. Lets the deploying user create and manage worker capacity.
iam:PassRole	On eks-cluster-role (the same role, trusted by both eks.amazonaws.com and ec2.amazonaws.com — see Section 5), so EKS can launch nodes using it.

Design note

- A single combined role (eks-cluster-role) is used for both the control plane and the node group, rather than the more common pattern of one role per purpose. Its trust policy lists both eks.amazonaws.com and ec2.amazonaws.com as principals so it works for both.

5. IAM Roles Used by the Cluster

These two roles are assumed by AWS services on the cluster's behalf — they are distinct from any human IAM user's own permissions.

5.1 eks-cluster-role

Field	Detail
Trusted principals	eks.amazonaws.com, ec2.amazonaws.com
Attached policies	AmazonEKSClusterPolicy AmazonEKSWorkerNodePolicy AmazonEKS_CNI_Policy AmazonEC2ContainerRegistryReadOnly
Used by	The EKS control plane and every node in the managed node group
Grants	Control-plane cluster management; kubelet node registration and lifecycle; pod networking via the VPC CNI; read-only access to pull container images from Amazon ECR.

5.2 eks-ebs-lb-role

Field	Detail
Trusted principals	pods.eks.amazonaws.com (EKS Pod Identity — no OIDC provider is configured for this cluster)
Attached policies	AmazonEBSCSIDriverPolicyV2 AmazonEBSCSIDriverPolicy Custom load-balancer-controller policy (Section 7)
Associated to	ebs-csi-controller-sa (kube-system) aws-load-balancer-controller (kube-system)
Grants	Provisioning/attaching/detaching Amazon EBS volumes for PersistentVolumeClaims, and creating/managing the Application Load Balancer, target groups, listeners and security groups for Ingress resources.

Design note

- This role is also shared across two unrelated addons (EBS CSI driver and the Load Balancer Controller) rather than split into two roles — the same combined-role pattern as eks-cluster-role.

6. EKS Add-ons (kube-system)

Add-on	Type	Purpose
vpc-cni	DaemonSet	Assigns a VPC-routable IP address to every pod.
kube-proxy	DaemonSet	Implements Kubernetes Service routing (including NodePort) on every node.
coredns	Deployment	Cluster-internal DNS resolution for Service names.
eks-pod-identity-agent	DaemonSet	Issues short-lived AWS credentials to pods associated with an IAM role via EKS Pod Identity.
aws-ebs-csi-driver	Controller + DaemonSet	Implements the Container Storage Interface for Amazon EBS, backing PersistentVolumeClaims.

7. AWS Load Balancer Controller & Application Load Balancer

Unlike the add-ons above, the AWS Load Balancer Controller is installed separately via Helm (chart v1.8.1, app v2.8.1) into kube-system, not as an EKS-managed add-on. It watches Kubernetes Ingress resources and provisions/manages a real AWS Application Load Balancer, target group and listener to match. Traffic flow: Internet → ALB (:80) → target group (EC2 instance, NodePort 30080) → kube-proxy → guacamole-web Service → pod.

WAF and Shield integration is explicitly disabled on this controller (`enableWaf=false`, `enableWafv2=false`, `enableShield=false`) since the workload does not use either — this also avoids needing `wafv2:*` / `shield:*` permissions.

Permissions required (attached to eks-ebs-lb-role)

Permission (abbreviated)	Why it's needed
ec2:Describe* (VPCs, subnets, security groups, instances, network interfaces, tags)	Discover the network topology to place the ALB correctly.
ec2:CreateSecurityGroup, AuthorizeSecurityGroupIngress, RevokeSecurityGroupIngress, CreateTags, DeleteTags	Create and maintain the security group that fronts the ALB.
elasticloadbalancing:CreateLoadBalancer, CreateTargetGroup, CreateListener, CreateRule	Provision the ALB, target group, listener and routing rule for each Ingress.
elasticloadbalancing:Describe*, Modify*, RegisterTargets, DeregisterTargets	Keep the ALB's targets in sync with healthy nodes and reconcile config drift.

Permission (abbreviated)	Why it's needed
<code>elasticloadbalancing:DeleteLoadBalancer, DeleteTargetGroup, DeleteListener, DeleteRule</code>	Clean up AWS resources when an Ingress is deleted.
<code>iam:CreateServiceLinkedRole</code> (condition: <code>elasticloadbalancing.amazonaws.com</code>)	One-time, account-wide prerequisite for Elastic Load Balancing to operate.

Not required for this deployment: `wafv2:GetWebACL*`, `wafv2:AssociateWebACL`, `shield:GetSubscriptionState` and related actions — these are only needed if WAF/Shield integration is enabled, which it deliberately is not here.

8. Kubernetes Workloads — namespace: guacamole

Kind	Name	Detail
Ingress	<code>guacamole-web</code>	<code>class: alb</code> · <code>healthcheck /guacamole</code> · triggers ALB provisioning (Section 7).
Service	<code>guacamole-web</code>	<code>NodePort 30080 → 8080</code> · <code>sessionAffinity: ClientIP</code> for 3h, so mid-session requests keep hitting the same pod.
Deployment	<code>guacamole-web</code>	Image <code>jilanigh/guacamole:redis-v1</code> · 3 replicas, autoscaled 3–9 by an HPA on 70% CPU · <code>topology-spread</code> + pod anti-affinity across nodes · <code>PodDisruptionBudget max-unavailable 1</code> · <code>readiness/liveness</code> probes on <code>/guacamole</code> .
Service + Deployment	<code>guacd</code>	Port 4822 · 1 replica · image <code>guacamole/guacd</code> · proxies RDP/VNC/SSH sessions on behalf of <code>guacamole-web</code> .
Service + Deployment	<code>postgres</code>	Port 5432 · 1 replica · image <code>postgres:15.3</code> · stores connections, users and session config.
PersistentVolumeClaim	<code>postgres-pvc</code>	5Gi, <code>storageClassName ebs-gp3</code> → provisioned as a real Amazon EBS gp3 volume by the EBS CSI driver.
StorageClass	<code>ebs-gp3</code>	<code>provisioner ebs.csi.aws.com</code> · <code>reclaimPolicy Retain</code> · <code>WaitForFirstConsumer</code> binding.
Service + Deployment	<code>redis</code>	Port 6379 · 1 replica · AOF persistence + password auth · shared session-token store so any <code>guacamole-web</code> pod can validate any user's session.
Job	<code>guacamole-init-db</code>	Runs once at deploy time to seed the Guacamole database schema.
CronJob	<code>postgres-backup</code>	Daily at 02:00 UTC · <code>pg_dump</code> , gzip, then uploaded via the S3 API to an external, non-AWS S3-compatible bucket.
Secrets	<code>postgres-secret</code> <code>redis-secret</code> <code>backup-s3-secret</code>	Database and cache credentials, and the external backup endpoint's access/secret keys.

These Kubernetes-native permissions are governed by the cluster's own RBAC, not by AWS IAM — creating them requires standard `kubectl`/cluster-admin access to the cluster's API server (e.g. via `aws eks update-kubeconfig` plus the EKS access entry granted at cluster creation time).

9. Consolidated Permissions Summary

Every AWS IAM permission referenced in this document, in one place. "Actor" is either the human/deploying IAM identity, or an AWS-assumed role belonging to the cluster itself.

Actor	Permission	Purpose
Deploying IAM identity	ec2:* (network CRUD: VPC, subnets, IGW, NAT, route tables, tags)	Provision the network layer (Section 2)
Deploying IAM identity	eks:CreateCluster / DescribeCluster / UpdateClusterConfig / UpdateClusterVersion / DeleteCluster / TagResource	Manage the EKS control plane (Section 3), scoped to the cluster ARN
Deploying IAM identity	eks:CreateNodegroup / DescribeNodegroup / UpdateNodegroupConfig / UpdateNodegroupVersion / DeleteNodegroup / ListNodegroups	Manage the node group (Section 4), scoped to the nodegroup ARN
Deploying IAM identity	iam:PassRole (condition: iam:PassedToService = eks.amazonaws.com)	Let EKS assume eks-cluster-role for the control plane and nodes
Deploying IAM identity	iam:CreateServiceLinkedRole (condition: iam:AWSServiceName = eks.amazonaws.com)	One-time, account-wide: first EKS cluster ever created in the account
eks-cluster-role	AmazonEKSClusterPolicy AmazonEKSWorkerNodePolicy AmazonEKS_CNI_Policy AmazonEC2ContainerRegistryReadOnly	Control plane + kubelet operation, pod networking, ECR image pulls
eks-ebs-lb-role	AmazonEBSCSIDriverPolicyV2 AmazonEBSCSIDriverPolicy	Provision/attach/detach EBS volumes for PVCs
eks-ebs-lb-role	Custom policy: ec2:Describe*, ec2:*SecurityGroup*, elasticloadbalancing:*	Create/manage the ALB, target groups, listeners and security groups for Ingress

10. How the Cluster Works, End to End

The sections above describe each component in isolation. This section walks through how they work together, following a request from a user's browser all the way down to the database, and describes everything running inside the cluster.

10.1 Request path: browser to database

1. A user's browser sends an HTTP request to the ALB's public DNS name on port 80.
2. The **Application Load Balancer**, sitting in the two public subnets, matches the request against its listener rule and forwards it to its target group.
3. The target group's targets are the EC2 worker nodes themselves (target-type: instance), on NodePort 30080. Whichever node receives the packet, its local **kube-proxy** uses the guacamole-web Service's routing rules to forward it to a specific pod — honouring sessionAffinity: ClientIP, so the same browser keeps landing on the same pod for 3 hours and doesn't lose its session mid-remote-desktop-session.
4. A **guacamole-web** pod (Tomcat running the Guacamole web application) handles the request. It reads/writes session tokens in **Redis** (so any of the 3–9 replica pods can validate a token issued by any other pod), and reads connection/user configuration from **PostgreSQL** over the internal Service address `postgres:5432`.
5. When the user opens an actual remote session, guacamole-web proxies the RDP/VNC/SSH protocol traffic through **guacd** (port 4822), which makes the real outbound connection to the target machine and streams it back over the browser's Guacamole session.

10.2 What keeps it running

1. The **EKS control plane** (API server, scheduler, controller-manager, etcd — all AWS-managed) is the source of truth for cluster state and decides where every pod runs.
2. The **scheduler** placed the guacamole-web pods across the 2 worker nodes using topology-spread and pod anti-affinity, so a single node failure never takes out more than about half the pods.
3. The **HorizontalPodAutoscaler** watches average CPU usage across the guacamole-web pods and scales the Deployment between 3 and 9 replicas as load changes.
4. The **PodDisruptionBudget** guarantees at most 1 guacamole-web pod is ever unavailable at once, whether that's from a rolling update, a node drain, or the autoscaler.
5. **kubelet** on each node keeps its pods running, restarts failed containers, and reports health back to the control plane via readiness/liveness probes on `/guacamole`.

10.3 Networking and storage glue

1. The **VPC CNI** add-on gives every pod a real, VPC-routable IP address, so pods can talk to each other and to AWS services directly.
2. **CoreDNS** resolves internal Service names (`postgres`, `redis`, `guacd`) to the right cluster IPs.
3. **kube-proxy** programs the networking rules on every node that make Services and NodePorts work.
4. The **AWS Load Balancer Controller** is the bridge between Kubernetes and AWS: it watches the guacamole-web Ingress object and is the thing that actually calls the AWS API to create and keep the ALB, target group and listener in sync with the cluster.

5. The **EBS CSI driver** is the bridge for storage: it watches the postgres-pvc PersistentVolumeClaim and is the thing that actually calls the AWS API to create and attach a real Amazon EBS gp3 volume to whichever node the postgres pod lands on.
6. Every night at 02:00 UTC, the **postgres-backup CronJob** spins up a short-lived pod that dumps the database, compresses it, uploads it to the external backup bucket over the S3 API, and then exits — it only exists for the duration of the backup.

10.4 Full component inventory

Layer	Components
Infrastructure	VPC, 2 public + 2 private subnets, Internet Gateway, NAT Gateway, route tables, Application Load Balancer, EBS gp3 volume
Cluster core	EKS control plane, managed node group (2–4 EC2 worker nodes)
IAM roles	eks-cluster-role, eks-ebs-lb-role
kube-system add-ons	vpc-cni, kube-proxy, coredns, eks-pod-identity-agent, aws-ebs-csi-driver, aws-load-balancer-controller
guacamole namespace — networking	Ingress (guacamole-web), Service (guacamole-web, NodePort)
guacamole namespace — compute	Deployments: guacamole-web (+HPA, +PDB), guacd, postgres, redis
guacamole namespace — storage	PersistentVolumeClaim (postgres-pvc), StorageClass (ebs-gp3)
guacamole namespace — jobs	Job (guacamole-init-db, one-time), CronJob (postgres-backup, daily)
guacamole namespace — config/secrets	Secrets: postgres-secret, redis-secret, backup-s3-secret

End of document.